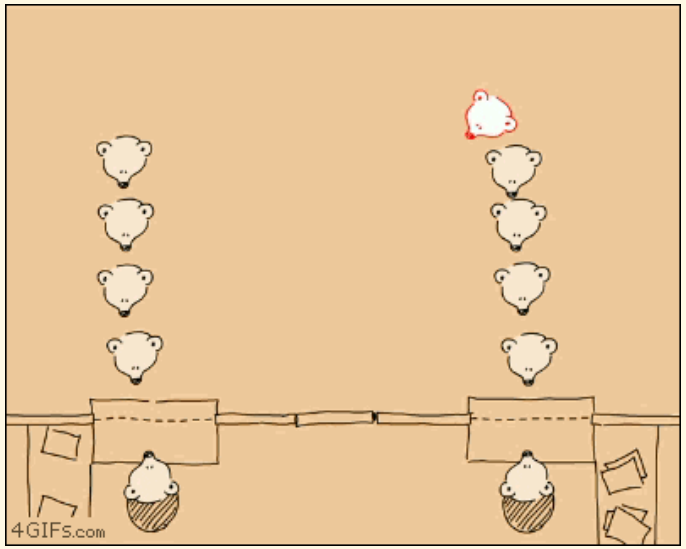
Stacks & Queues

- Stacks
- Queues
- Separation of interface and implementation

Overview

- Access restriction
  - Arrays: random
  - Linked lists: sequential
  - Stacks and Queues: limited

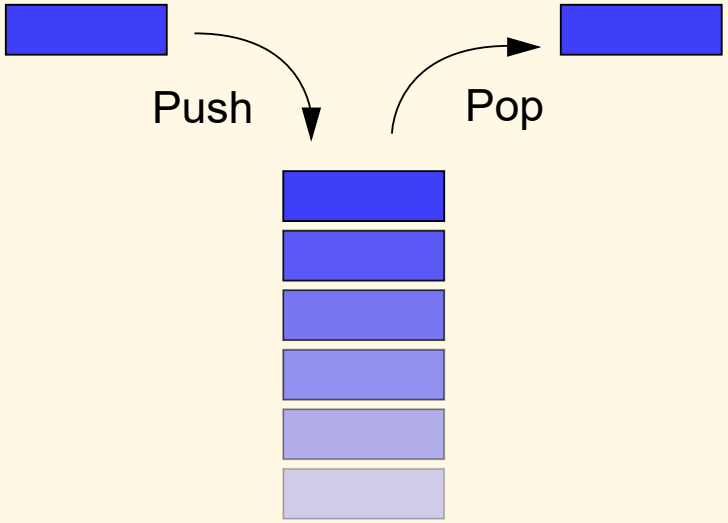
Stack



Queue

Stacks

What's a stack?



- Linear data structure which can only be accessed at one of its ends
- LIFO (last in, first out) basis
- Two ways of implementation:
  - Using array (static or dynamic)
  - Using linked list (dynamic)

Operations

- `create()`: creates a stack
- `destroy()`: destroys a stack
- `push()`: inserts an element
- `pop()`: removes last inserted element (optionally returns the element)
- `top()`: returns the last inserted element without removing it
- `isEmpty()`: checks if the stack is empty
- `isFull()`: checks if the stack is full

Static implementation

- Definitions

```
typedef int STACK_DATA;
#define MAX_ELEMENTS 20

typedef struct {
  STACK_DATA elements[MAX_ELEMENTS];
  int top;
} STACK;
```

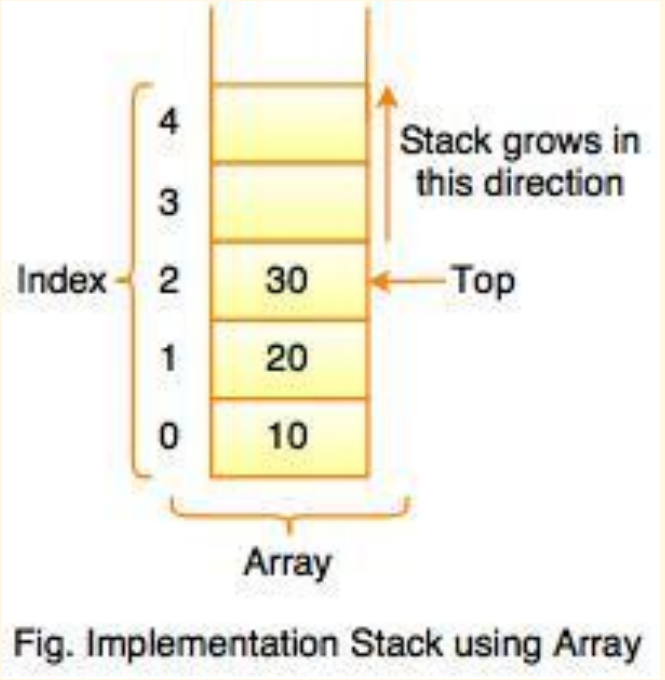


Fig. Implementation Stack using Array

- Initialization

```
void stInit(STACK* s) {
  s->top = -1;
}
```

Push

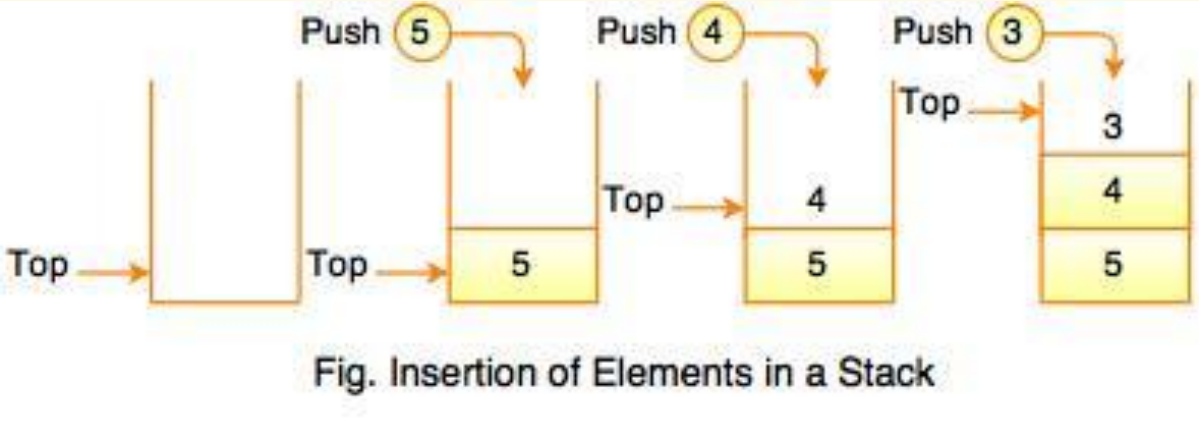


Fig. Insertion of Elements in a Stack

```
void stPush(STACK* s, STACK_DATA data) {
     if (stIsFull(s)) return;

     s->top++;
     s->elements[s->top] = data;
}
```

Pop

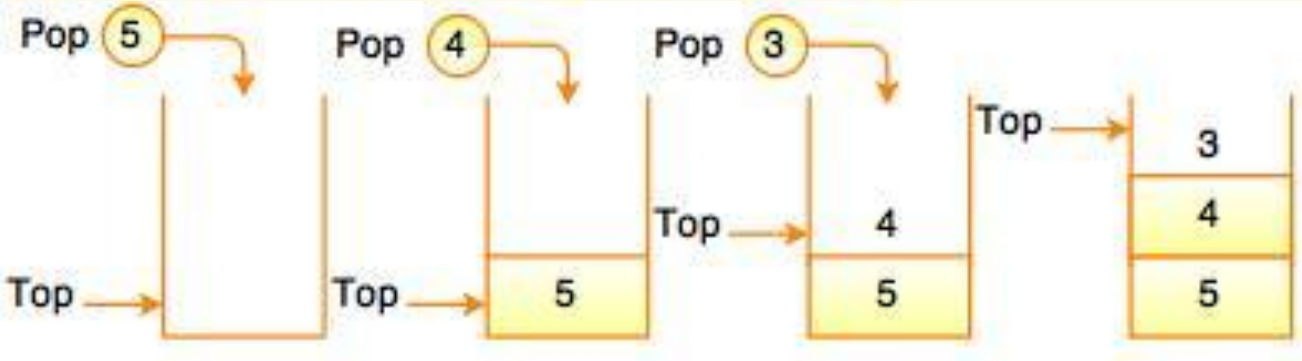


Fig. Deletion of Elements in a Stack

```
void stPop(STACK* s) {
     if (stIsEmpty(s)) return;

     s->top--;
}
```

Get top value

```
STACK_DATA stTop(STACK* s) {
     return s->elements[s->top];
}
```

Is empty? Is full?

```
int stIsEmpty(STACK* s) {
     return s->top == -1;
}


int stIsFull(STACK* s) {
     return s->top == MAX_ELEMENTS - 1;
}
```

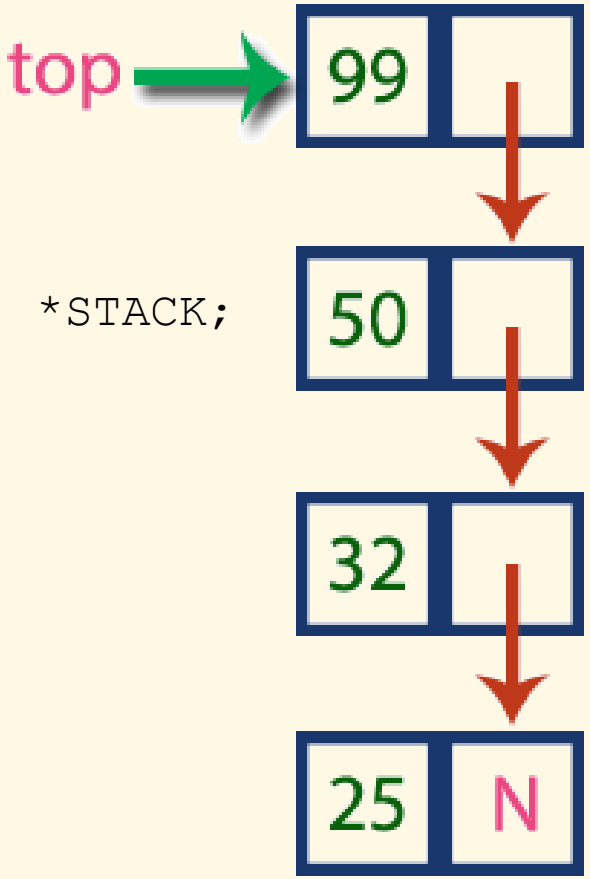
Clear all

```
void stDestroy(STACK* s) {
     s->top = -1;
}
```

Dynamic implementation

- Definitions

```
typedef int STACK_DATA;

struct _STACK_ELEM;
typedef struct _STACK_ELEM STACK_ELEM, *STACK;

struct _STACK_ELEM {
     STACK_DATA data;
     STACK_ELEM* next;
};
```

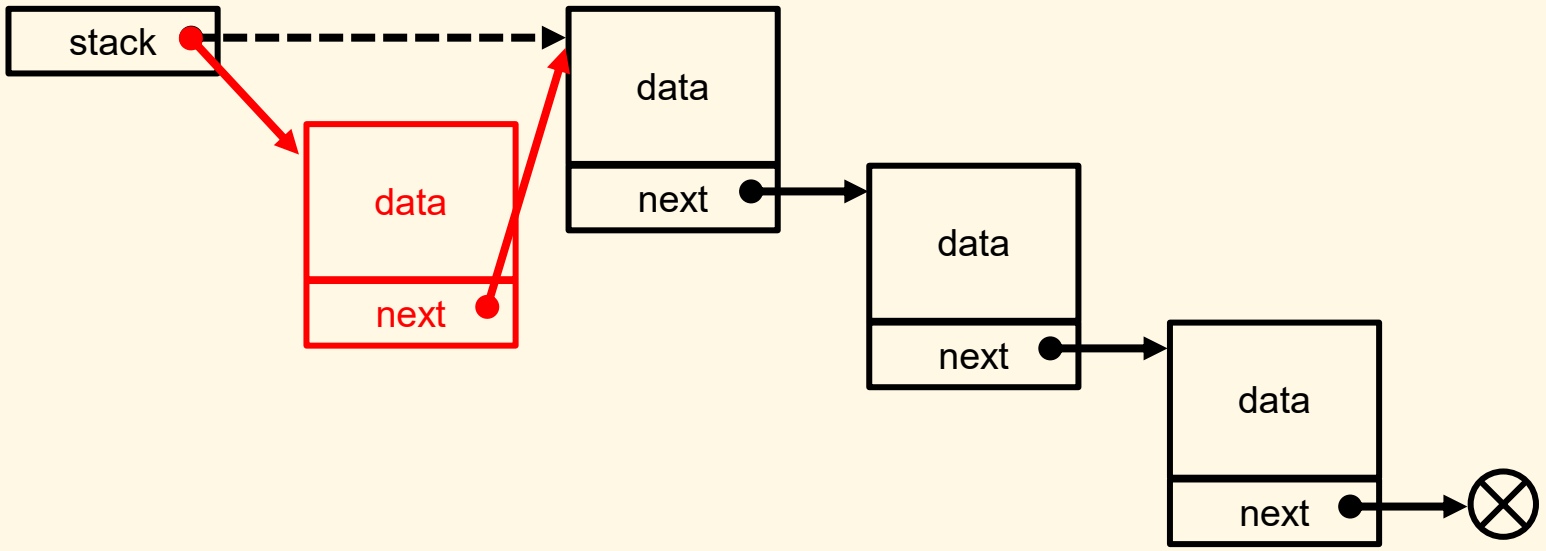


- Initialization

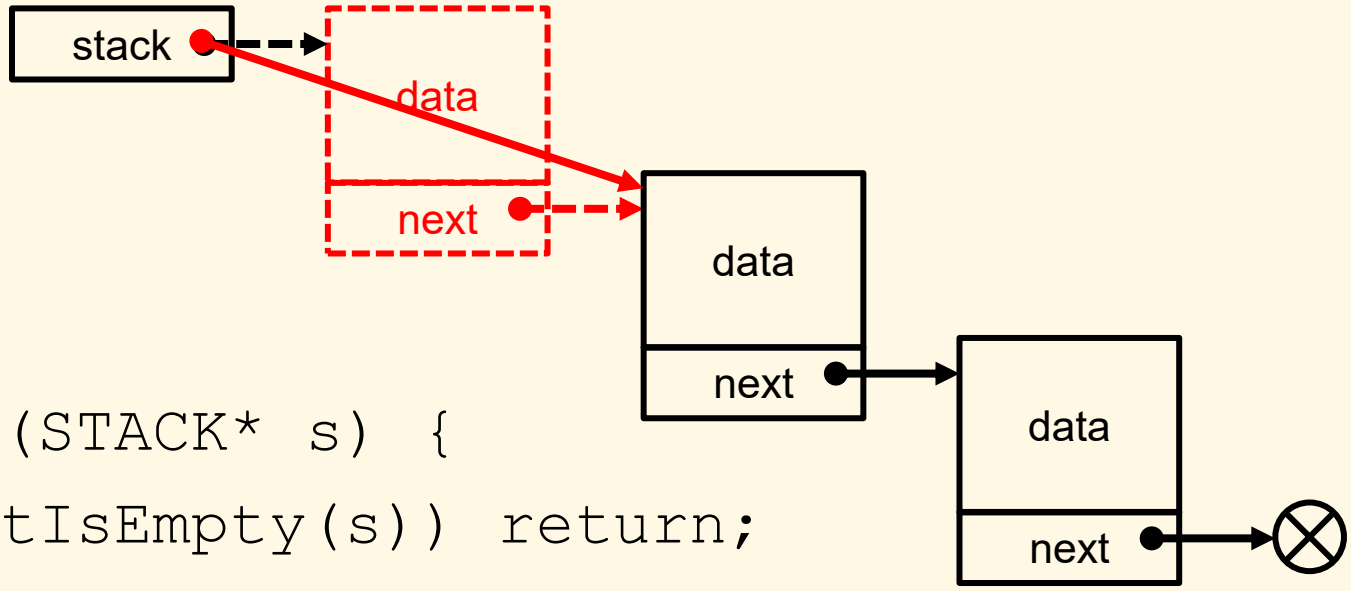
```
void stInit(STACK* s) {
     *s = NULL;
}
```

Push

```
void stPush(STACK* s, STACK_DATA data) {
        STACK_ELEM* p = (STACK_ELEM*)malloc(
                                        sizeof(STACK_ELEM));
        p->data = data;
        p->next = *s;

        *s = p;
}
```



Pop




```
void stPop(STACK* s) {
     if (stIsEmpty(s)) return;

     STACK_ELEM* p = *s;
     *s = (*s)->next;
     free(p);
}
```

Get top value

```
STACK_DATA stTop(STACK* s) {
     return (*s)->data;
}
```

Is empty? Is full?

```
int stIsEmpty(STACK* s) {
     return *s == NULL;
}


int stIsFull(STACK* s) {
     return 0;
}
```

Clear all

```
void stDestroy(STACK* s) {
     while (*s) {
          STACK_ELEM* p = *s;
          *s = (*s)->next;
          free(p);
     }
}
```

Applications of stacks

- Memory representation for function calls in computer program execution
- Undo operation in editors
- History in web browsers
- Alternative implementation of recursive functions
- Math expression evaluation (reverse Polish notation)

Separation of interface and implementation

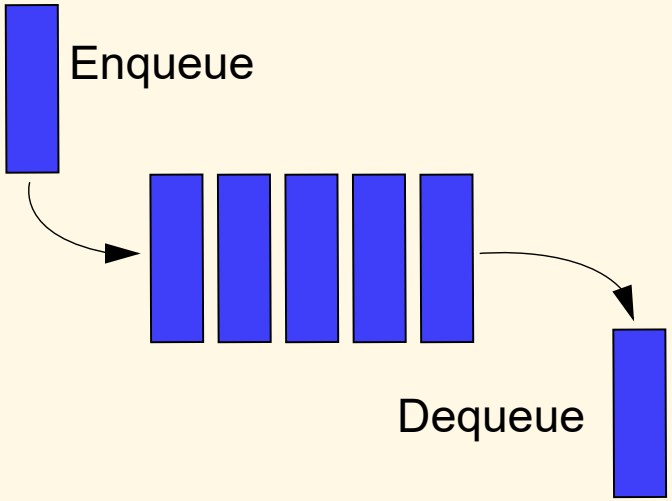
Overview

- Motivation:
  - A problem can have different methods/mechanisms in implementation, but they share a same or similar usage
  - Reducing implementation detail exposed to the user
  - Reducing physical coupling

- C & C++ languages allow to separate the interface (.h files) from its implementation (.c/.cpp files)

Queues

What's a queue?

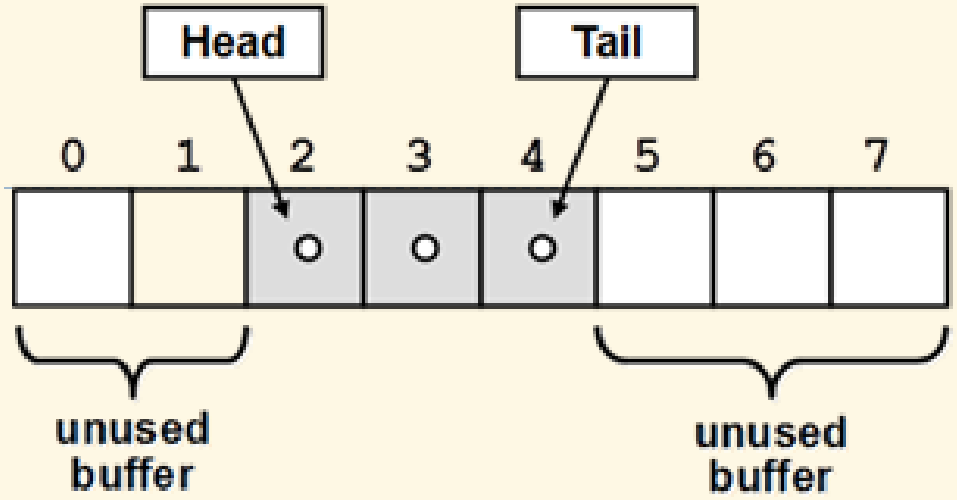


- Linear data structure which can only be accessed at its two ends
- FIFO (first in, first out) basis
- Two ways of implementation:
  - Using array (static or dynamic)
  - Using linked list (dynamic)

Operations

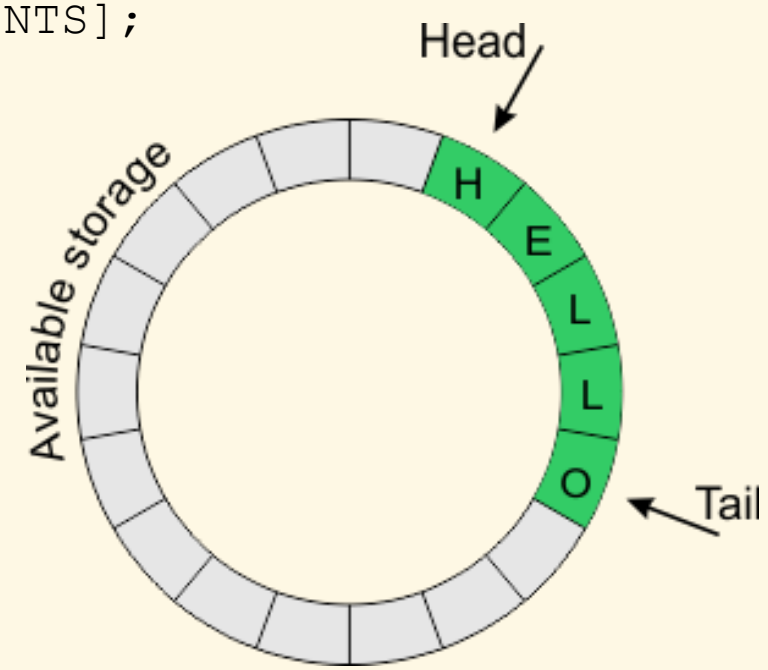
- `create()`: creates a queue
- `destroy()`: destroys a queue
- `enqueue()`: inserts an element
- `dequeue()`: removes first inserted element (optionally returns the element)
- `top()`: returns the first inserted element without removing it
- `isEmpty()`: checks if the queue is empty
- `isFull()`: checks if the queue is full

Static implementation



- Definitions

```
typedef int QUEUE_DATA;
#define MAX_ELEMENTS 20

typedef struct {
  QUEUE_DATA elements[MAX_ELEMENTS];
  int head;
  int count;
} QUEUE;
```

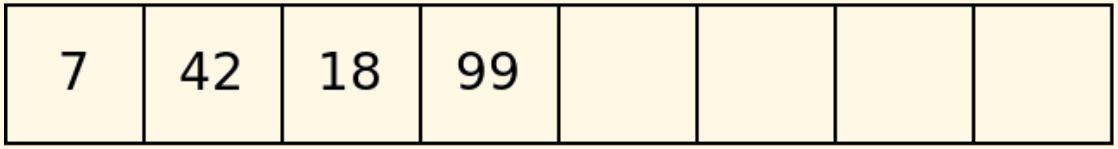


- Initialization

```
void qInit(QUEUE* s) {
  s->head = 0;
  s->count = 0;
}
```

Enqueue

enqueue 7
enqueue 42
enqueue 18
enqueue 99

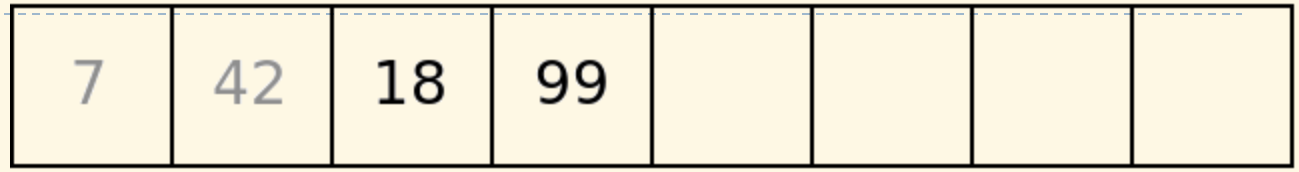


Start = 0
End = 3

```
void qEnqueue(QUEUE* s, QUEUE_DATA data) {
      if (qIsFull(s)) return;

      int i = (s->head + s->count) % MAX_ELEMENTS;
      s->elements[i] = data;

      s->count++;
}
```

Dequeue

dequeue
dequeue



Start = 2
End = 3

```
void qDequeue(QUEUE* s) {
      if (qIsEmpty(s)) return;

      s->count--;

      if (++ s->head == MAX_ELEMENTS)
            s->head = 0;
}
```

Get first value

```
QUEUE_DATA qHead(QUEUE* s) {
     return s->elements[s->head];
}
```

Is empty? Is full?

```
int qIsEmpty(QUEUE* s) {
     return s->count == 0;
}


int qIsFull(QUEUE* s) {
     return s->count == MAX_ELEMENTS;
}
```

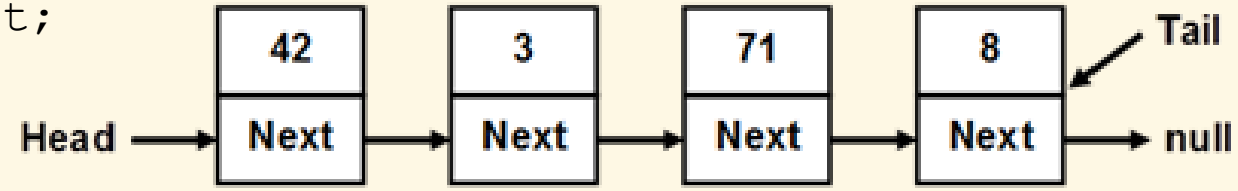
Clear all

```
void qDestroy(QUEUE* s) {
     s->head = 0;
     s->count = 0;
}
```

Dynamic implementation

- Definitions

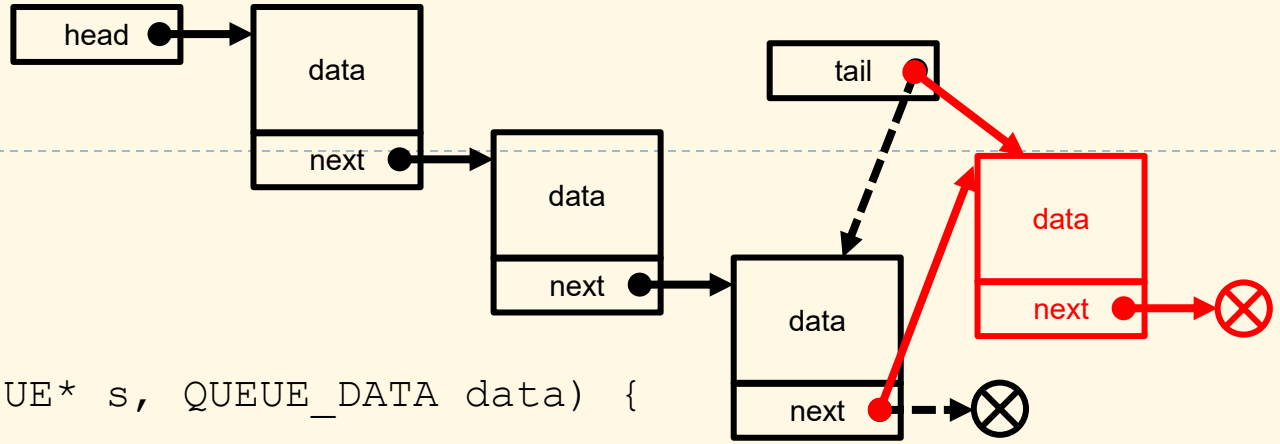
```
typedef int QUEUE_DATA;

struct _QUEUE_ELEM;
typedef struct _QUEUE_ELEM QUEUE_ELEM;

struct _QUEUE_ELEM {
  QUEUE_DATA data;
  QUEUE_ELEM* next;
};

typedef struct {
  QUEUE_ELEM* head;
  QUEUE_ELEM* tail;
} QUEUE;
```



Initialization

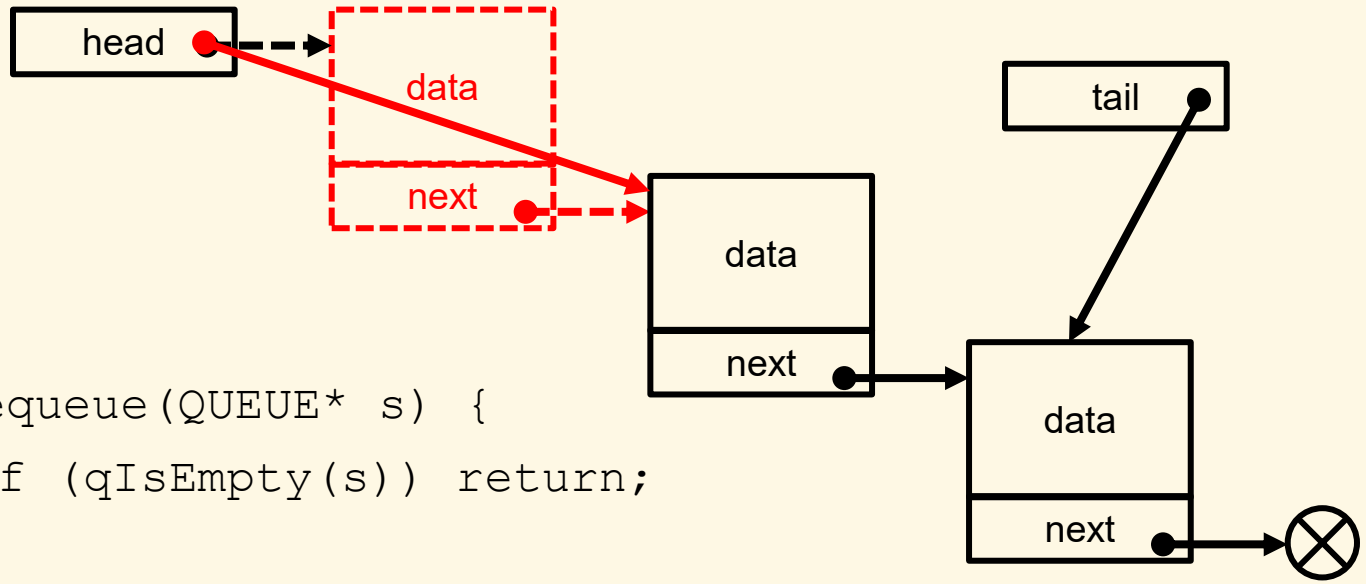
```
void stInit(QUEUE* s) {
  s->head = s->tail = NULL;
}
```

Enqueue




```
void qEnqueue(QUEUE* s, QUEUE_DATA data) {
        if (qIsFull(s)) return;

        QUEUE_ELEM* p = (QUEUE_ELEM*)malloc(
                        sizeof(QUEUE_ELEM));
        p->data = data;
        p->next = NULL;

        if (s->tail) s->tail->next = p;
        s->tail = p;

        if (!s->head) s->head = p;
}
```

Dequeue




```
void qDequeue(QUEUE* s) {
        if (qIsEmpty(s)) return;

        QUEUE_ELEM* p = s->head;
        s->head = p->next;
        free(p);

        if (!s->head) s->tail = NULL;
}
```

Get first value

```
QUEUE_DATA qHead(QUEUE* s) {
     return s->head->data;
}
```

Is empty? Is full?

```
int qIsEmpty(QUEUE* s) {
     return s->head == NULL;
}


int qIsFull(QUEUE* s) {
     return 0;
}
```

Clear all

```
void qDestroy(QUEUE* s) {
     while (s->head) {
          QUEUE_ELEM* p = s->head;
          s->head = p->next;
          free(p);
     }

     s->head = NULL;
     s->tail = NULL;
}
```

Applications of queues

- Data transfer and communication buffer
- Reading/writing buffer
- Implementation of client/server services

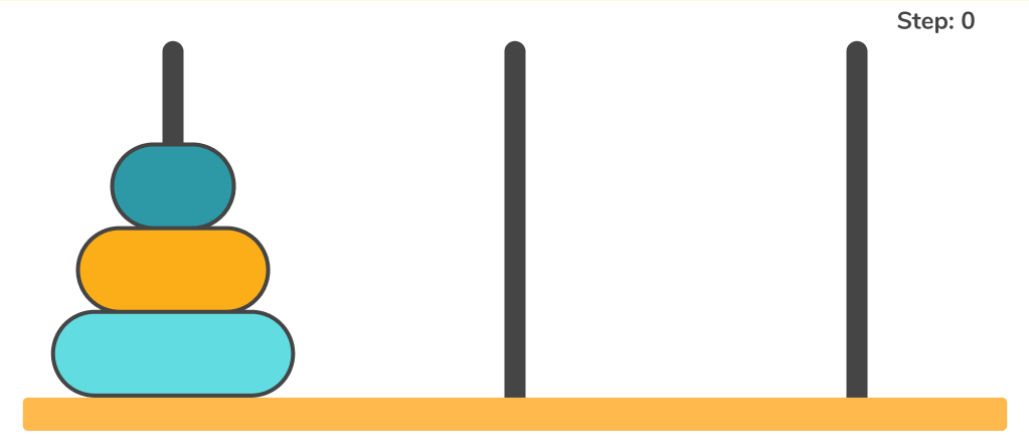
Priority Queues

Priority queue

- A type of queue that arranges elements based on their priority values
  - Elements with higher priority values are typically retrieved before elements with lower priority values
  - The priority value is usually stored in a member variable, but can also be a function of the element data
- Several implementation methods:
  - Using an unsorted array or linked list
  - Using a sorted array or linked list
  - Using a heap
  - Using a binary search tree

Homework: Stack

1. Write a program which reads integers and prints them in normal and reversed order.
2. Implement the reverse Polish (postfix) notation for simple expression evaluation.
3. The Tower of Hanoi is a puzzle consisting of 3 rods and n disks. The task is to move all disks to another rod, but:
   - Only the uppermost disk can be moved
   - Larger disks cannot be placed on top of smaller disks



Write a function to solve the problem with help of a stack, given the number of disks n.

Homework: Queue

1. Write a program to simulate a task producer-consumer work queue.
2. Implement a priority queue with help of a static array.
3. Implement a priority queue with help of a sorted linked list.